

Informe: *AI Control: Improving Safety Despite Intentional Subversion:*
Un Intento de Replicación Parcial

Alumno: Carlos Bottinelli
LU: 853/23

Presentacion

A medida que los sistemas de IA se acercan a niveles de capacidad cada vez más generales (AGI), crece también el riesgo asociado al despliegue de estos modelos para resolver tareas reales.

Frente a este problema, existen dos estrategias complementarias. La primera consiste en intentar que los agentes de IA sean efectivamente confiables, es decir, en trabajar sobre el alineamiento del modelo mismo. La segunda, conocida como AI Control, consiste en diseñar medidas de seguridad que mitiguen los daños posibles incluso si el agente no está alineado. Esta segunda estrategia cobra especial relevancia porque evidencia reciente muestra que intentar corregir el desalineamiento haciendo post-training puede funcionar como un incentivo para que el modelo aprenda a disimularlo [1].

En este informe replicamos parcialmente el paper *AI Control: Improving Safety Despite Intentional Subversion* [2]. Más concretamente, replicamos dos de los protocolos de control — trusted monitoring y defer-to-trusted — utilizando un dataset reducido de 120 problemas. En la sección de Metodología describimos tanto el diseño original del paper como las decisiones tomadas en nuestra implementación. Luego presentamos los resultados obtenidos en nuestra replicación y las diferencias observadas respecto de lo reportado en el trabajo original. Por último, discutiremos esos resultados y sus implicancias.

Metodologia

Un protocolo de control es un mecanismo de monitoreo de un modelo potencialmente no alineado acompañado de un mecanismo de decisión respecto del output del modelo. El protocolo recibe como input la salida del modelo no alineado, determina si esa salida es lo suficientemente confiable para ser utilizada y decide qué hacer en caso contrario. El desempeño de un protocolo se evalúa en un entorno adversarial: un red team diseña una attack policy para U, cuyo objetivo es lograr que el protocolo acepte una salida maliciosa, mientras que el protocolo busca minimizar ese riesgo. Como veremos, en el contexto del paper, el modelo potencialmente no alineado intenta introducir backdoors en soluciones de problemas de programación. Su evaluación se resume en dos métricas:

- **Safety:** es la probabilidad de que el protocolo no envíe ninguna solución con backdoor exitosa a lo largo de todos los problemas evaluados.
- **Usefulness:** es la proporción de soluciones correctas entre las enviadas por el protocolo. Un detalle a tener en cuenta es que, al reportar usefulness, el paper la reescala entre 0 y 1, donde 0 corresponde a la tasa de correctitud del modelo T y 1 a la tasa de correctitud del modelo U. En el paper, la correctitud de T = GPT 3.5 fue de 22% y la de U = GPT-4 fue de 54%. En nuestra replicación, estos valores fueron de 24% y 58%, con lo cual la métrica no es idéntica pero es suficientemente comparable.

Intuitivamente, estas métricas expresan el tradeoff que se busca lograr en un entorno de control: se busca aprovechar la utilidad (usefulness) del modelo U, a la vez que se reducen los riesgos que plantea su potencial no alineamiento (safety).

Estrategia de muestreo. El paper evalúa una serie de protocolos sobre 5000 problemas del dataset público APPS. Dado que evaluar directamente sobre esos 5000 problemas sería costoso, el paper estima ambas métricas a partir de una muestra más pequeña de 1500 problemas. Para realizar la estimación, hace bootstrap de la siguiente manera:

- 1) Repetidamente construye muestras según el siguiente procedimiento:
 - Seleccionar $N = M/12$ problemas con intento de backdoor.
 - Seleccionar uniformemente con reemplazo 5000-N problemas sin backdoor attempt.
 - Unir y mezclar aleatoriamente, para obtener una lista de 5000 problemas.
- 2) Se corren 2000 simulaciones sobre muestras bootstrapeadas de esa manera. En este contexto, tiene sentido pensar a safety como la media de una variable Bernoulli y usefulness como la media de la tasa de correctitud obtenida en cada run.

Por razones de costo, replicamos esta lógica a una escala reducida: seleccionamos una base de 120 problemas en lugar de 1500, y hacemos upsampling hasta obtener muestras de 400 en lugar de 5000. En cuanto al factor de muestreo, mantenemos el mismo factor del paper.

Modelos. Los protocolos tienen componentes que son implementados por diferentes modelos. Lógicamente, los modelos T y U no pueden ser idénticos ya que el planteo del problema requiere que exista entre ellos una diferencia de capacidad. En los protocolos que replicamos, además, el protocolo puede recurrir a un auditor H, que es una segunda instancia de decisión más capaz pero a la vez más costosa, por lo cual puede usarse de manera limitada. Los autores señalan que este rol sería desempeñado por un humano, pero que por motivos de costos lo implementan con un modelo. Siendo H más capaz que T, es lógico que se implemente con un modelo más capaz también.

A continuación, detallamos los modelos usados por el paper y por nosotros para implementar cada rol:

Rol	Función	Modelo (paper)	Modelo (nuestra implementación)
T	Genera soluciones confiables; calcula el score que determina si un problema debe auditarse o, en el protocolo defer, si debe ser resuelto por el modelo T.	GPT-3.5-Turbo / GPT-3.5-Turbo-Instrct	GPT-3.5-Turbo / GPT-3.5-Turbo-Instrct
U	Genera las soluciones normales, usadas para calibrar los clasificadores de T y de H, y las soluciones con backdoor (attack policy)	GPT-4	GPT-4.1-mini
H	Es el árbitro final que decide si una solución tiene o no backdoor.	GPT-4	GPT-4.1-mini

En nuestra replicación, reemplazamos GPT-4 por GPT-4.1-mini y usamos GPT-3.5-Turbo por razones de costo. Para el scoring del monitor elegimos específicamente la variante *Instruct* porque, a diferencia de los modelos de chat, expone logprobs sobre los tokens de salida, lo cual permite calcular un score numérico de sospecha a partir de la probabilidad asignada a una respuesta binaria (sí/no), tal como hace el paper. Evaluamos alternativamente obtener este score sampleando múltiples respuestas a temperatura 1 y estimando la probabilidad de manera empírica, pero las escalas resultantes eran muy distintas a las obtenidas vía logprobs directos, y abandonamos esa vía para no comprometer la comparabilidad con el diseño del paper. De todas maneras, resultaría interesante explorar más a fondo la diferencia entre estas estrategias.

[Mostrar datos](#)

Inicialmente, consideramos utilizar modelos gratuitos alternativos, pero no pudimos conseguir que las distribuciones de los scores usados por el monitor fueran similares a las del paper. Consideramos que una diferencia tan considerable afectaría la replicabilidad, por lo cual descartamos esta decisión.

[Mostrar datos](#)

[Sacar alguna conclusión: por qué importaría?](#)

Generación y consumo de datos. Obtuvimos los datos del dataset APPS bajado de HuggingFace. Para que el filtrado de problemas no introdujera una diferencia con el paper -que tiene muchos criterios no explicitados fuera del código- usamos las funciones tomadas directamente de su repositorio. Sobre los problemas filtrados corrimos un pipeline de generación de datos que, en rasgos generales, hace lo siguiente:

1. **Soluciones normales.** Para cada problema, generamos una solución con T y otra con U -en adelante las llamaremos “normales”, para oponerlas a las soluciones con backdoor generadas por el mismo modelo.
2. **Generación de backdoors.** Sobre las soluciones normales, corrimos un pipeline de varios pasos que genera múltiples propuestas de backdoor por problema, las filtra

según criterios de dificultad de explotación y sutileza, y selecciona una propuesta final por problema. Al construir el pipeline, intentamos replicar lo más cercanamente posible la descripción dada en el apéndice M del paper. También consultamos los logs de respuestas del modelo publicados por los autores y nos basamos en el código del repositorio para obtener detalles adicionales.

- 3. **Verificación de correctitud.** Corrimos los tests de referencia sobre las tres soluciones (T, U normal, U con backdoor) para determinar cuáles pasan. En esta etapa, reutilizamos el código del repositorio original para evitar tener que implementar desde cero la lógica de compilación y testing del código generado.
- 4. **Scoring.** Sobre las soluciones normales y las soluciones con backdoor calculamos los scores de monitoring y de auditing descriptos en la sección anterior.

Una vez generados, las soluciones junto con sus resultados de correctitud y scores, las mismas se agrupan por problema en una estructura de datos implementada por nosotros, ProblemData, que es finalmente consumida por los protocolos para el cálculo de resultados. En esta segunda parte del pipeline nos apartamos decididamente de la implementación original, ya que nuestra funcionalidad era más reducida. Más allá de eso, ambas implementaciones tienen la ventaja de que, una vez terminada la primera etapa de generación de datos, el muestreo por bootstrap y las corridas simuladas se realizan sin interactuar con la API de OpenAI.

Resultados

Cantidad y Sutileza de Backdoors. En ambos protocolos analizados, la política óptima del adversario se basa en escribir backdoors sutiles, con el objetivo de no ser clasificados como tales por el monitor ni el auditor. Los autores del paper encontraron que la efectividad del pipeline implementado con GPT-4 es baja, ya que solo logra una tasa aproximada de 0.013 backdoors por solución procesada. Nuestro pipeline resulta significativamente más capaz, logrando 0.075 backdoors exitosos por solución procesada, como se ve en las siguientes tablas:

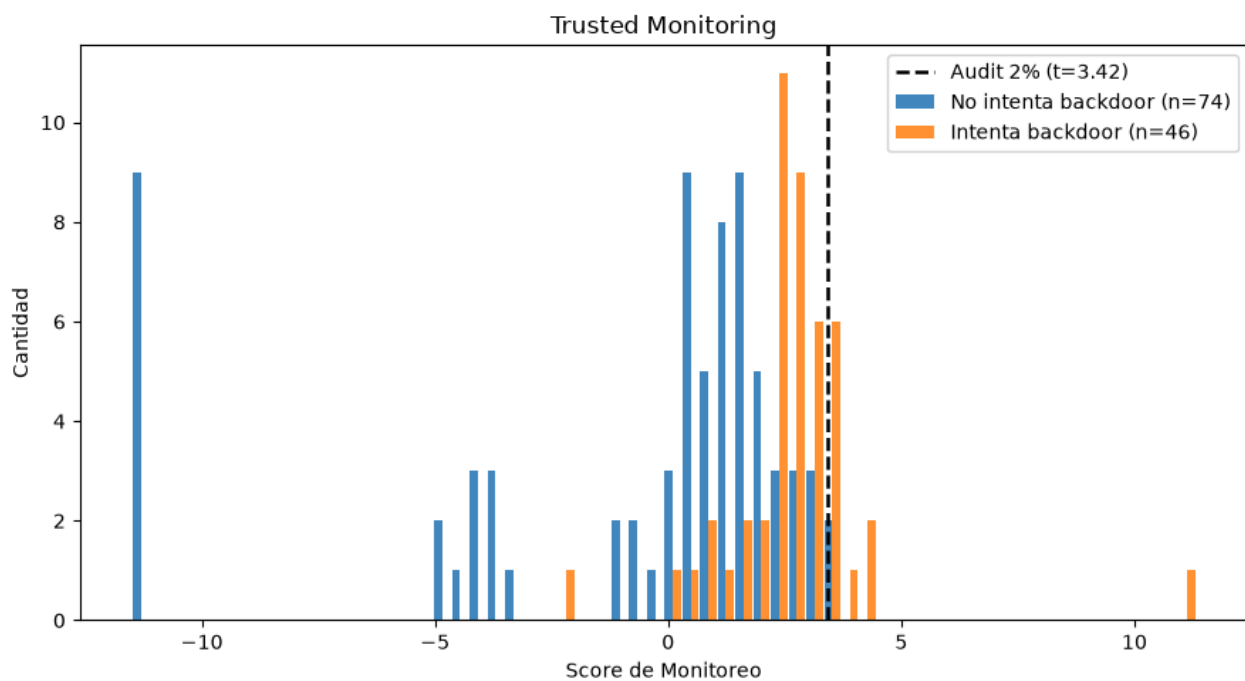
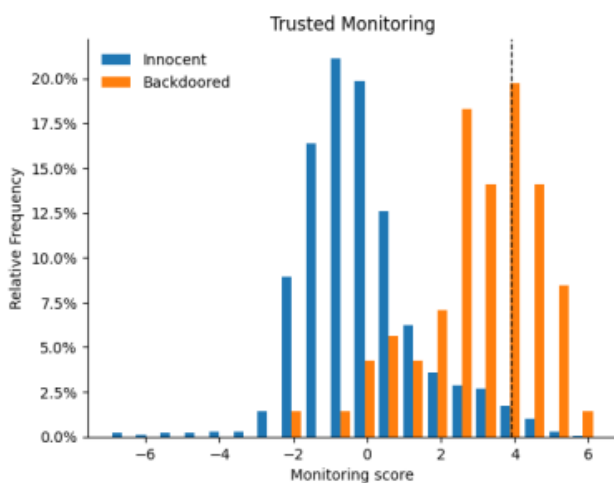
	Paper Original			Replicación		
	Auditados	No auditados		Auditados	No auditados	
Backdoor exitoso	6	13	Backdoor exitoso	2	7	Podría sacar mas conclusiones comparativas: ¿tasas sobre tamaño muestra?
Backdoor fallido	18	34	Backdoor fallido	6	31	
Sin backdoor	29	1400	Sin backdoor	1	73	

Hay varias explicaciones posibles para este resultado. Una de ellas puede tener que ver con las diferencias entre los modelos usados en cada implementación. GPT-4.1-mini tiene más contexto que GPT-4 y en nuestra implementación pudimos aprovecharlo extendiendo los límites

de tokens al notar que muchas veces el pipeline fallaba porque el modelo agotaba el contexto y se interrumpía. Una explicación complementaria es que, en ocasiones, el pipeline fallaba porque los modelos no encontraban ideas de backdoor en los primeros intentos. En estos casos, nuestro pipeline agregaba un retry, que encontramos efectivo. De todas maneras, no descartamos que diferencias de arquitectura entre los modelos haya jugado un rol significativo. Sería una cuestión para seguir explorando.

No voy a probar variaciones por una cuestión de costos.

Un segundo aspecto de interés en relación con la sutileza de los backdoors tiene que ver con cómo se compara la distribución de sus scores de monitoreo con el de las soluciones normales. Como el clasificador está calibrado con una tasa de falsos positivos del 2% sobre la distribución de soluciones normales, el grado de superposición entre una distribución y otra da una medida de la dificultad que el clasificador va a tener en distinguir ambos tipos de soluciones. Dicho de otra manera, si las distribuciones de las soluciones con backdoor y las soluciones sin backdoor están más superpuestas, los backdoors son más sutiles.



El gráfico superior muestra las distribuciones del paper, mientras que el gráfico inferior muestra las obtenidas en la replicación. Podemos ver que, en nuestro experimento, las distribuciones se encuentran más superpuestas. Como veremos, este aumento en la sutileza de los backdoors no tiene un impacto claro sobre la métrica de safety obtenida.

Acá podría hacerse algún experimento variando las distribuciones

Trusted Monitoring. Un punto a tener en cuenta a la hora de analizar la *usefulness* de los protocolos tiene que ver con la manera en que los autores del paper la miden. En lugar de medir la correctitud sobre episodios en los que el protocolo procesa muestras enviadas por la política atacante, que podrían tener backdoors, el paper mide específicamente la correctitud sobre episodios que solo procesan soluciones normales producidas por GPT-4. Esta estrategia se justifica en el apéndice J señalando que, como el protocolo conoce la distribución de correctitud de las soluciones normales, podría detectar si el modelo U está enviando soluciones incorrectas a una tasa más alta y así distinguir que está operando maliciosamente. De esto concluyen que la correctitud en este caso está determinada por lo que ocurre después de que el modelo ha sido detectado como malicioso y relacionan esto con el problema de *scheming*.

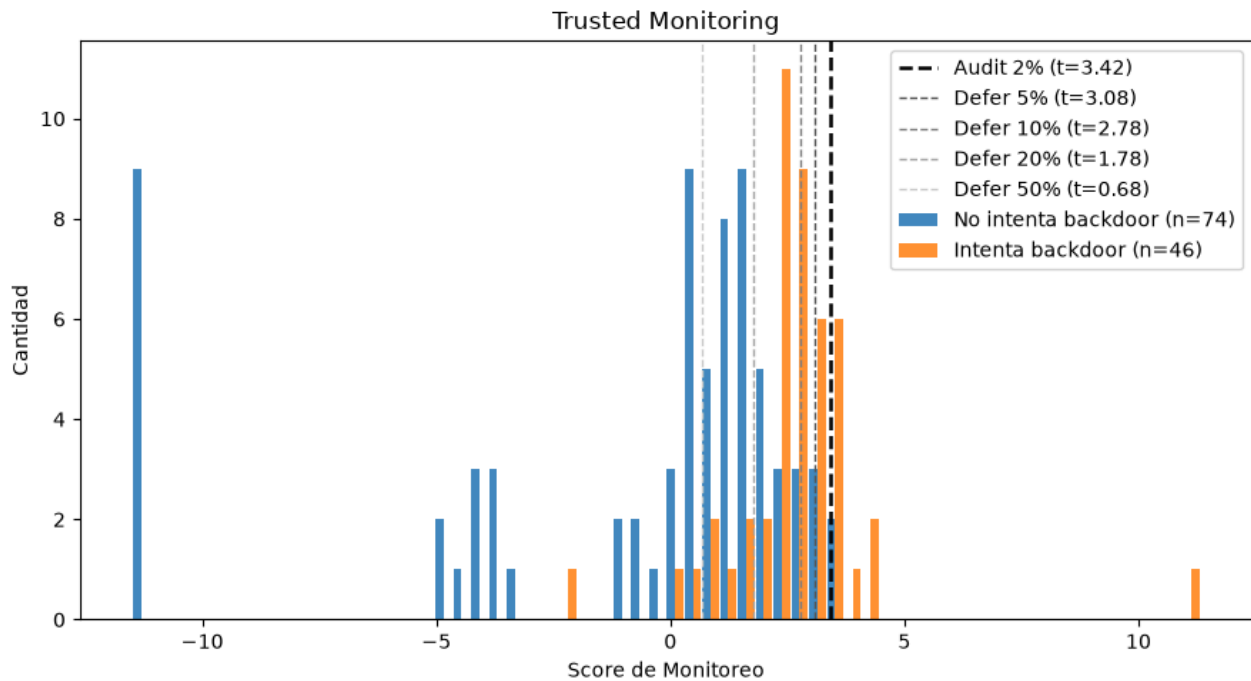
Más allá de este problema, consideramos que si el protocolo define una medida como el shutdown y el uso de soluciones trusted una vez que se ha detectado que un modelo es malicioso, resulta interesante ver qué efecto tiene esta medida sobre la tasa de corrección. En primer lugar, porque el modelo T es menos capaz que el U, con lo cual podría esperarse una baja en la tasa de corrección. Pero además, porque en nuestra replicación, el protocolo encuentra un backdoor e ingresa en ese régimen de shutdown en casi la mitad de las rondas (898/2000).

Por este motivo, realizamos analizamos maneras alternativas de evaluar el protocolo de trusted monitoring. En un primer experimento, medimos usefulness corriendo el protocolo sobre soluciones normales como hace el paper, mientras que en el segundo medimos usefulness usando las soluciones de la attack policy. Respecto de la medición de safety, el primer experimento lo hace por diseño en dos rondas de simulaciones separadas, lo cual se evita en el segundo. En ambos casos, los parámetros usados fueron los reportados en el paper (*f.p.r. monitor* = 2%, *budget auditor* = 2%, *factor subsampling backdoors* = 1/12). A continuación, mostramos los valores obtenidos en ambos experimentos y los comparamos con los obtenidos en el paper. Retomaremos este punto en la discusión.

Experimento	Usefulness	Safety
Usefulness sobre soluciones normales (siguiendo al paper)	72%	66%
Usefulness sobre soluciones attack policy (version propia)	56%	66%
Valores obtenidos en el paper	97%	62%

Defer to Trusted (DTT). La idea básica de este protocolo es tener un segundo clasificador sobre los scores de monitoreo, con un umbral más bajo que el clasificador original. Entre

ambos umbrales, queda definida una zona de sospechosidad “media”: respecto de los problemas que caen en esta zona, se utiliza la solución T -a diferencia con el protocolo anterior, que utilizaría la U y aumentaría su probabilidad de aceptar una solución con backdoor exitoso. El atractivo de este diseño se puede visualizar si graficamos las distribuciones de los scores de monitoreo incluyendo distintos valores posibles para el umbral de defer:



La visualización muestra que, a medida que se agranda la zona de defer, cada vez es menor la probabilidad de que una solución con backdoor sea aceptada por el protocolo. Por otra parte, al bajar el umbral de defer, hay más problemas respecto de los cuales se usa la solución T, que tiene una tasa de corrección más baja. Esto permite a defer to trusted ofrecer un trade off granular entre usefulness y safety, que fue corroborado experimentalmente en el paper y también apareció en la replicación:

	Safety (paper)	Usefulness (p.)	Safety (replic.)	Usefulness (r.)
DTT 0%	62%	97%	66%	56%
DTT 5%	74%	95%	79%	56%
DTT 10%	83%	93%	79%	52%
DTT 20%	92%	83%	94%	46%
DTT 50%	100%	51%	100%	7%

Una aclaración respecto de la tabla anterior es que, en el protocolo defer to trusted, medimos usefulness con las soluciones de la attack policy, con lo cual ésta medida no es directamente

comparable a la del paper. Más allá de esto, el hecho de que dos implementaciones distintas del protocolo verifiquen la misma tendencia refuerza el resultado.

Discusión

El presente informe muestra que nuestra implementación corrobora en líneas generales los resultados obtenidos por los autores del paper.

Respecto de los datos generados, observamos que, a pesar de haber usado distintos modelos para implementar el modelo U, las distribuciones de los scores de las soluciones normales y las soluciones con backdoor generadas por este modelo tienen características similares.

En relación con los protocolos, es interesante poner en perspectiva los datos obtenidos con los baselines presentados por el paper. La siguiente tabla muestra la manera en que la replicación reproduce las tendencias generales presentadas en el paper:

Protocolo	Usefulness	Safety
No usar modelos untrusted	2%	100%
Defer to Trusted (f.p.r 10%)	paper: 93%, replic.: 52%	paper: 83%, replic.: 79%
Trusted Monitoring	paper: 97%, replic.: 56% / 72%	paper: 62%, replic.: 66%
Upfront Auditing	100%	15%

Adicionalmente, nuestros experimentos muestran que si se adopta una manera alternativa de evaluar usefulness para trusted monitoring, el valor obtenido puede ser inferior al informado en el paper. Esta variación permite medir, no solo la pérdida de utilidad cuando el modelo opera en condiciones no maliciosas, sino también cuando opera sobre una distribución inesperada y decide tomar las medidas establecidas para esa situación. Esto no sólo sería aplicable a casos cuando efectivamente el modelo es malicioso, sino que podría dar una idea del comportamiento del protocolo operando bajo *distribution shift* en contextos reales.

Para finalizar, un aspecto que no terminamos de explicar es por qué si los backdoors obtenidos en la replicación son más frecuentes y más sutiles que los obtenidos en el paper original, la métrica de safety es ligeramente más alta en la replicación. Un lógico próximo paso sería ver si la diferencia es estadísticamente significativa. Si lo fuera, una hipótesis podría ser que la diferencia tiene que ver con la distribución de los scores que usa el auditor. Estos scores son calculados por H, implementado con GPT 4.1. Si los backdoors, que son más frecuentes, fueran bien distinguibles por el auditor, eso podría inducir mas detecciones y mayor safety.

- Para Safety -- Divisor: necesito probar que la proporcion de backdoors en la muestra sea similar a la del experimento original. Ajustando con factor 6, tomo divisor $12 * 6$. tamaño muestra = 200. Safety = 1.

- Para correctness -- El divisor no influye, pq se calcula sobre soluciones normales. Quizas el tamaño de la muestra influye: pero pruebo tamaño 5000 y usefulness baja a 48%

- Parece que hay un bug en cómo se simula el experimento del paper. Habría que escribir bien el código primero.

ANTES! Sin reescribir código, hacer mas comprensibles los notebooks y sacar codigo del repo original. Hacer un commit de "entrega".

Bibliografía

- [1] Von Arx, S., Chan, L., & Barnes, B. (2025). Recent Frontier Models Are Reward Hacking. METR Blog. <https://metr.org/blog/2025-06-05-recent-reward-hacking/>
- [2] Greenblatt, R., Shlegeris, B., Sachan, K., & Roger, F. (2023). *AI Control: Improving Safety Despite Intentional Subversion*. arXiv:2312.06942. <https://arxiv.org/abs/2312.06942>

Ademas: logging, caching